

# ACME Runtime — On-Chain Generative Art Runtime

Technical specification and artist guide for the ACME on-chain runtime system. Version 1.0.0 | Asset: `ACMERUNTIME010`

---

## Table of Contents

1. Overview
  2. Architecture
  3. On-Chain Assets
  4. Runtime API Reference
  5. Seed System
  6. Library Loading
  7. P5.js Integration
  8. THREE.js Integration
  9. Canvas API (No Library)
  10. SVG Mode
  11. Traits and Attributes
  12. Dynamic Component Import
  13. Bootstrap Loader
  14. Bridge Script (Preview Mode)
  15. Minting Flow
  16. Common Patterns
  17. Troubleshooting
  18. Full Runtime Source Code
- 

## Overview

The ACME Runtime is a shared JavaScript library minted as an on-chain ACME asset (`ACMERUNTIME010`). Every piece of generative art in the ACME ecosystem loads this runtime rather than bundling its own copy.

**What it provides:** - Deterministic seeded pseudo-random number generator (SFC32) - Seed resolution from multiple sources (URL, Cortex metadata, asset name derivation) - Trait system for NFT attribute definition - Dynamic library loading with gzip decompression - Dynamic component imports for recursive/composable art - Asset identity detection (parent/child/token number parsing)

**Why on-chain:** - Every minted artwork shares one canonical runtime (4KB) instead of each inlining ~3KB - Updates to the runtime benefit all art retroactively - Smaller per-mint storage cost on Bitcoin

---

## Architecture

### Before (Inline)

Minted HTML (~5KB+ per piece)  
  Inline runtime (~3KB): PRNG, seed resolution, library loader  
  Library loading code (~500B)  
  Artist code

### After (Shared Runtime)

Minted HTML (~1.5KB per piece)  
  Bootstrap loader (~500B): fetches ACMERuntime010  
  Artist code

ACMERuntime010 (shared, loaded once)  
  Seed resolution (URL → Cortex → SHA-256 → random)  
  SFC32 PRNG  
  Trait system  
  acme.loadLibrary() with gzip decompression  
  acme.import() for component composition

### Execution Flow

Browser loads minted HTML

Bootstrap loader executes

```
fetch("/api/assets/ACMERuntime010/content")  
Detect gzip → decompress via DecompressionStream  
Inject <script> → window.acme available
```

Runtime self-initializes

```
Detect asset ID from URL  
Parse parent/child/tokenNumber  
Resolve seed (URL param → Cortex tag → child-derived → random)  
Initialize PRNG  
Resolve acme.ready promise
```

(Optional) Library loads

```

acme.loadLibrary("P5JSMIN190", { globalVar: "p5" })
fetch + gzip decompress + <script> inject
window.p5 available

```

Artist code executes

### Preview vs. Minting

| Aspect    | Preview (Studio)                                 | Minted (On-Chain)                           |
|-----------|--------------------------------------------------|---------------------------------------------|
| Runtime   | Bridge script (inline, ~4KB)                     | Bootstrap →<br>ACMERUNTIME010               |
| Libraries | <script<br>src="/api/assets/P5JSMIN190/content"> | acme.loadLibrary("P5JSMIN190")              |
| Console   | Captured via postMessage<br>to Studio            | Native browser console                      |
| FPS       | Tracked and reported to<br>Studio                | Not tracked                                 |
| Seed      | Provided by Studio's<br>seedStore                | Resolved by runtime<br>(URL/Cortex/derived) |

### On-Chain Assets

#### Required Assets

| Asset ID        | Content                       | Size (gzip) | Global Variable |
|-----------------|-------------------------------|-------------|-----------------|
| ACMERUNTIME010  | ACME<br>Runtime<br>v1.0.0     | ~1.8 KB     | window.acme     |
| P5JSMIN190      | P5.js v1.9.0<br>minified      | ~240 KB     | window.p5       |
| THREEJSMIN01580 | THREE.js<br>r158 UMD<br>build | ~161 KB     | window.THREE    |

#### Minting an Asset

Each asset is a standard ACME asset with the Cortex tag `x:generative-library`:

```

{
  "tag": "x:generative-library",
  "value": {

```

```

    "name": "ACME Runtime",
    "version": "1.0.0",
    "globalVar": "acme",
    "compressed": true
  }
}

```

#### Steps to mint ACMERUNTIME010:

1. Take the minified runtime source (4.1 KB)
2. Optionally gzip compress it (reduces to ~1.8 KB)
3. Mint as ACME asset with name ACMERUNTIME010
4. Content-type: `application/javascript`
5. Add Cortex tag `x:generative-library` with the metadata above

#### Steps to mint P5JSMIN190:

1. Download `p5.min.js` v1.9.0 from the P5.js releases
2. Gzip compress: `gzip -9 -c p5.min.js > p5.min.js.gz` (~240 KB)
3. Mint as ACME asset with name P5JSMIN190
4. Content-type: `application/javascript`
5. Cortex tag: `{ "name": "P5.js", "version": "1.9.0", "globalVar": "p5", "compressed": true }`

#### Steps to mint THREEJSMIN01580:

1. Download `three.min.js` r158 UMD build (NOT the ES module — the UMD/IIFE build that sets `window.THREE`)
2. Gzip compress: `gzip -9 -c three.min.js > three.min.js.gz` (~161 KB)
3. Mint as ACME asset with name THREEJSMIN01580
4. Content-type: `application/javascript`
5. Cortex tag: `{ "name": "THREE.js", "version": "0.158.0", "globalVar": "THREE", "compressed": true }`

**Important:** `THREE.js` must be the UMD build that exposes `window.THREE`, NOT the ES module version. On-chain HTML cannot use `import` statements or import maps.

---

## Runtime API Reference

Once the runtime loads, `window.acme` exposes the following API:

### Core

**`acme.hash`** — **string** The deterministic seed for this artwork. A 66-character hex string starting with `0x`.

```
console.log(acme.hash)
// "0x4a7f2b9c1e8d3f6a5b0c7d4e9f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f"
```

**acme.random()** — **number** Returns a deterministic pseudo-random number between 0 (inclusive) and 1 (exclusive), seeded by **acme.hash**. Uses the SFC32 algorithm — the same PRNG used by fxhash and Art Blocks.

```
// Always produces the same sequence for the same hash
let r1 = acme.random() // e.g., 0.7234...
let r2 = acme.random() // e.g., 0.1891...
let r3 = acme.random() // e.g., 0.4502...
```

**Why not Math.random()? Math.random()** produces different output every page load. **acme.random()** is deterministic — the same seed always produces the same artwork. This is essential for on-chain generative art where collectors expect reproducible output.

**acme.ready** — **Promise<acme>** A promise that resolves when the runtime has finished initialization (seed resolved, PRNG ready). Always **await** this before using **acme.hash** or **acme.random()**.

```
acme.ready.then(() => {
  console.log("Seed:", acme.hash)
  console.log("First random:", acme.random())
})
```

## Asset Identity

**acme.assetId** — **string | null** The ACME asset ID, auto-detected from the URL. For example, if the HTML is served from `/api/assets/MYART/content`, then **acme.assetId** is "MYART".

**acme.parentAsset** — **string | null** The parent portion of a hierarchical asset name. For "COLLECTION.42", this is "COLLECTION".

**acme.childName** — **string | null** The child portion after the dot. For "COLLECTION.42", this is "42".

**acme.tokenNumber** — **number | null** The child portion parsed as an integer. For "COLLECTION.42", this is 42. Returns **null** if the child isn't numeric or doesn't exist.

```
// Asset: MYART.137
acme.assetId      // "MYART.137"
acme.parentAsset  // "MYART"
acme.childName    // "137"
acme.tokenNumber  // 137
```

## Traits

**acme.trait(name, value)** — **void** Define a runtime trait for this artwork. Traits are key-value pairs that describe the artwork's characteristics (color palette, density, style, etc.).

```
acme.trait("Palette", "Neon Sunset")
acme.trait("Density", acme.random() > 0.5 ? "Dense" : "Sparse")
acme.trait("Shape Count", Math.floor(acme.random() * 100))
```

**acme.attributes** — **Array<{ trait\_type: string, value: any }>** NFT attributes loaded from the asset's Cortex metadata. These are set at mint time and are immutable.

**acme.getTraits()** — **object** Returns a merged map of Cortex attributes and runtime-defined traits.

```
const all = acme.getTraits()
// { "Palette": "Neon Sunset", "Density": "Dense", "Shape Count": 73, ... }
```

## Dynamic Loading

**acme.loadLibrary(assetUrl, opts?)** — **Promise<any>** Fetch a library from on-chain, automatically decompress if gzipped, and inject as a `<script>` tag.

```
// Load by ACME asset ID
await acme.loadLibrary("P5JSMIN190", { globalVar: "p5" })
// window.p5 is now available

// Load by direct URL
await acme.loadLibrary("/api/assets/MY_CUSTOM_LIB/content", { globalVar: "MyLib" })
```

**Parameters:** - **assetUrl** — ACME asset ID (e.g., "P5JSMIN190") or a URL path - **opts.globalVar** — the global variable the library exposes (used to return the library object)

**How it works:** 1. Fetches the content as **ArrayBuffer** 2. Checks first 2 bytes for gzip magic (0x1f 0x8b) 3. If gzipped: decompresses via **DecompressionStream** 4. Creates a `<script>` element and injects the source 5. Returns **window[globalVar]** if specified

Results are cached — subsequent calls with the same asset return immediately.

**acme.import(assetId)** — **Promise<any>** Dynamically import a JavaScript component from on-chain. The component is treated as a CommonJS module (**module.exports**).

```
const colorUtils = await acme.import("MYCOLORUTILS")
const palette = colorUtils.generatePalette(acme.random())
```

**How it works:** 1. Fetches the asset's content as text 2. Wraps it in a CommonJS → ES module adapter 3. Creates a Blob URL and uses `dynamic import()` 4. Returns `module.exports`

This enables composable, recursive generative art where one piece can import utilities from another on-chain asset.

---

## Seed System

The seed (`acme.hash`) determines the entire output of a generative artwork. The runtime resolves it through a priority chain:

### 1. URL Parameters (Highest Priority)

```
https://example.com/art?hash=0xABCD...
https://example.com/art?seed=0xABCD...
https://example.com/art?tokenHash=0xABCD...
```

This is how the ARTCORE Studio passes seeds during preview.

### 2. Cortex Metadata Tag

If the asset has a Cortex tag `x:generative-seed`, that value is used:

```
{
  "tag": "x:generative-seed",
  "value": "0x4a7f2b9c1e8d3f6a5b0c7d4e9f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f"
}
```

### 3. SHA-256 Derivation (Subassets)

For subassets like `COLLECTION.42`, if no explicit seed is set, the runtime derives one deterministically:

```
seed = SHA-256("COLLECTION.42")
```

This means every token in a collection gets a unique but deterministic seed based on its name. Token 42 always gets the same seed, token 43 gets a different one.

### 4. Random Fallback

If none of the above sources provide a seed, `crypto.getRandomValues()` generates a random 32-byte hex string. This only happens during development or when viewing art outside the ACME ecosystem.

---

## P5.js Integration

P5.js (v1.9.0) is loaded from on-chain asset P5JSMIN190. It's the most popular framework for 2D generative art.

### Basic P5.js with ACME Seed

```
// Use acme.random() instead of random() for determinism
function setup() {
  createCanvas(windowWidth, windowHeight);
  colorMode(HSB, 360, 100, 100, 100);
  background(0);

  // Seed P5's noise with the ACME hash for deterministic Perlin noise
  noiseSeed(parseInt(acme.hash.slice(2, 10), 16));

  // Define traits based on the seed
  let palette = acme.random() > 0.5 ? "Warm" : "Cool";
  acme.trait("Palette", palette);
}

function draw() {
  // Your art code using acme.random() for all randomness
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}
```

### P5.js Flow Field Example (Deterministic)

```
let particles = [];
let flowfield;
let cols, rows;
let scale = 20;
let zoff = 0;
let baseHue;

function setup() {
  createCanvas(windowWidth, windowHeight);
  colorMode(HSB, 360, 100, 100, 50);
  background(0);

  // Use acme seed for deterministic noise
  noiseSeed(parseInt(acme.hash.slice(2, 10), 16));

  // Derive visual properties from the seed
```



```

baseHue = Math.floor(acme.random() * 360);
let particleCount = Math.floor(acme.random() * 300) + 200;
let speed = acme.random() * 2 + 1;

// Record traits
acme.traits("Base Hue", baseHue);
acme.traits("Particle Count", particleCount);
acme.traits("Speed", speed.toFixed(1));

cols = floor(width / scale);
rows = floor(height / scale);
flowfield = new Array(cols * rows);

for (let i = 0; i < particleCount; i++) {
  particles.push({
    pos: createVector(acme.random() * width, acme.random() * height),
    vel: createVector(0, 0),
    acc: createVector(0, 0),
    maxSpeed: speed,
    prevPos: null,
    hue: baseHue + acme.random() * 60 - 30,
  });
  particles[i].prevPos = particles[i].pos.copy();
}
}

function draw() {
  // Generate flow field from Perlin noise
  let yoff = 0;
  for (let y = 0; y < rows; y++) {
    let xoff = 0;
    for (let x = 0; x < cols; x++) {
      let index = x + y * cols;
      let angle = noise(xoff, yoff, zoff) * TWO_PI * 2;
      flowfield[index] = p5.Vector.fromAngle(angle).setMag(1);
      xoff += 0.1;
    }
    yoff += 0.1;
  }
  zoff += 0.003;

  // Update and draw particles
  for (let p of particles) {
    let x = floor(p.pos.x / scale);
    let y = floor(p.pos.y / scale);
    let force = flowfield[constrain(x + y * cols, 0, flowfield.length - 1)];
  }
}

```

```

    if (force) p.acc.add(force);

    p.vel.add(p.acc);
    p.vel.limit(p.maxSpeed);
    p.pos.add(p.vel);
    p.acc.mult(0);

    stroke(p.hue % 360, 80, 100, 10);
    strokeWeight(1);
    line(p.pos.x, p.pos.y, p.prevPos.x, p.prevPos.y);
    p.prevPos = p.pos.copy();

    // Wrap edges
    if (p.pos.x > width) { p.pos.x = 0; p.prevPos = p.pos.copy(); }
    if (p.pos.x < 0) { p.pos.x = width; p.prevPos = p.pos.copy(); }
    if (p.pos.y > height) { p.pos.y = 0; p.prevPos = p.pos.copy(); }
    if (p.pos.y < 0) { p.pos.y = height; p.prevPos = p.pos.copy(); }
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
  cols = floor(width / scale);
  rows = floor(height / scale);
  flowfield = new Array(cols * rows);
}

```

## P5.js Best Practices for On-Chain Art

1. **Always use `acme.random()` instead of `random()`** — P5's `random()` is non-deterministic
2. **Always call `noiseSeed()`** — Without it, Perlin noise differs per load
3. **Always include `windowResized()`** — The canvas must adapt to its container
4. **Avoid `Math.random()`** — Use `acme.random()` everywhere
5. **No external URLs** — All resources must be on-chain or generated in code
6. **No `Date.now()` or `new Date()`** — These make output non-deterministic
7. **Define traits with `acme.trait()`** — Collectors can filter/search by traits

---

## THREE.js Integration

THREE.js (r158) is loaded from on-chain asset `THREEJSMIN01580` as a UMD build that sets `window.THREE`.

**Critical:** The on-chain THREE.js is a UMD build, NOT an ES module. Do NOT use `import * as THREE from 'three'`. Use `window.THREE` directly — it's available as a global after loading.

### Basic THREE.js with ACME Seed

```
// THREE is available as a global (UMD build)
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x111111);

const camera = new THREE.PerspectiveCamera(
  75, window.innerWidth / window.innerHeight, 0.1, 1000
);
camera.position.z = 5;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
document.body.appendChild(renderer.domElement);

// Use acme.random() for deterministic object placement
const count = Math.floor(acme.random() * 20) + 5;
const hueBase = acme.random();

acme.trait("Object Count", count);
acme.trait("Hue Base", (hueBase * 360).toFixed(0));

for (let i = 0; i < count; i++) {
  const geometry = new THREE.IcosahedronGeometry(
    acme.random() * 0.5 + 0.2, // radius
    Math.floor(acme.random() * 3) // detail
  );
  const color = new THREE.Color();
  color.setHSL(hueBase + acme.random() * 0.2, 0.7, 0.6);

  const material = new THREE.MeshStandardMaterial({
    color,
    roughness: 0.3,
    metalness: 0.8,
  });

  const mesh = new THREE.Mesh(geometry, material);
  mesh.position.set(
    (acme.random() - 0.5) * 8,
    (acme.random() - 0.5) * 8,
    (acme.random() - 0.5) * 8
  );
}
```

```

    );
    mesh.rotation.set(
      acme.random() * Math.PI * 2,
      acme.random() * Math.PI * 2,
      acme.random() * Math.PI * 2
    );
    scene.add(mesh);
  }

  // Lighting
  const dirLight = new THREE.DirectionalLight(0xffffff, 1);
  dirLight.position.set(5, 5, 5);
  scene.add(dirLight);
  scene.add(new THREE.AmbientLight(0x404040, 0.5));

  // Animation
  function animate() {
    requestAnimationFrame(animate);
    scene.rotation.y += 0.003;
    renderer.render(scene, camera);
  }
  animate();

  // Handle resize
  window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
  });

```

### THREE.js Generative Particle Cloud

```

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x000008);

const camera = new THREE.PerspectiveCamera(
  75, window.innerWidth / window.innerHeight, 0.1, 1000
);
camera.position.z = 50;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);

// Deterministic particle cloud
const particleCount = Math.floor(acme.random() * 5000) + 2000;

```

```

const spread = acme.random() * 50 + 50;
const colorMode = acme.random() > 0.5 ? "galaxy" : "nebula";

acme.trait("Particles", particleCount);
acme.trait("Spread", spread.toFixed(0));
acme.trait("Color Mode", colorMode);

const geometry = new THREE.BufferGeometry();
const positions = new Float32Array(particleCount * 3);
const colors = new Float32Array(particleCount * 3);

for (let i = 0; i < particleCount * 3; i += 3) {
    // Spherical distribution with density falloff
    const r = Math.pow(acme.random(), 0.5) * spread;
    const theta = acme.random() * Math.PI * 2;
    const phi = Math.acos(2 * acme.random() - 1);

    positions[i] = r * Math.sin(phi) * Math.cos(theta);
    positions[i + 1] = r * Math.sin(phi) * Math.sin(theta);
    positions[i + 2] = r * Math.cos(phi);

    if (colorMode === "galaxy") {
        colors[i] = acme.random() * 0.3 + 0.7; // R
        colors[i + 1] = acme.random() * 0.3; // G
        colors[i + 2] = acme.random() * 0.5 + 0.5; // B
    } else {
        colors[i] = acme.random() * 0.5;
        colors[i + 1] = acme.random() * 0.5 + 0.5;
        colors[i + 2] = acme.random() * 0.3 + 0.7;
    }
}

geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geometry.setAttribute('color', new THREE.BufferAttribute(colors, 3));

const material = new THREE.PointsMaterial({
    size: 0.3,
    vertexColors: true,
    transparent: true,
    opacity: 0.8,
    blending: THREE.AdditiveBlending,
});

const particles = new THREE.Points(geometry, material);
scene.add(particles);

```

```

let time = 0;
function animate() {
  requestAnimationFrame(animate);
  time += 0.0005;
  particles.rotation.y = time;
  particles.rotation.x = Math.sin(time * 0.7) * 0.1;
  renderer.render(scene, camera);
}
animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});

```

### THREE.js Best Practices for On-Chain Art

1. **UMD build only** — Use new `THREE.Scene()`, not `import { Scene } from 'three'`
2. **Use `acme.random()` for ALL randomness** — `Math.random()` is non-deterministic
3. **Always handle resize** — Add `window.addEventListener('resize', ...)`
4. **Set pixel ratio cap** — `renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))`
5. **No external textures** — Generate all textures procedurally or use data URIs
6. **Dispose geometries/materials** — Prevent WebGL memory leaks in long-running animations
7. **No OrbitControls from CDN** — Either inline the code or keep it non-interactive

---

### Canvas API (No Library)

For lightweight art that doesn't need P5.js or THREE.js. No library is loaded — just the ACME runtime.

#### Deterministic Canvas Art

```

const canvas = document.createElement('canvas');
document.body.appendChild(canvas);
const ctx = canvas.getContext('2d');

function resize() {

```

```

    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
  }
  resize();
  window.addEventListener('resize', resize);

  // Wait for ACME runtime to be ready
  acme.ready.then(() => {
    const bgHue = Math.floor(acme.random() * 360);
    const lineCount = Math.floor(acme.random() * 50) + 20;
    const style = acme.random() > 0.5 ? "curved" : "straight";

    acme.trait("Background Hue", bgHue);
    acme.trait("Line Count", lineCount);
    acme.trait("Style", style);

    ctx.fillStyle = `hsl(${bgHue}, 20%, 5%)`;
    ctx.fillRect(0, 0, canvas.width, canvas.height);

    for (let i = 0; i < lineCount; i++) {
      const hue = (bgHue + acme.random() * 120) % 360;
      ctx.strokeStyle = `hsla(${hue}, 70%, 60%, 0.6)`;
      ctx.lineWidth = acme.random() * 3 + 0.5;

      ctx.beginPath();
      const x1 = acme.random() * canvas.width;
      const y1 = acme.random() * canvas.height;

      if (style === "curved") {
        const cp1x = acme.random() * canvas.width;
        const cp1y = acme.random() * canvas.height;
        const cp2x = acme.random() * canvas.width;
        const cp2y = acme.random() * canvas.height;
        const x2 = acme.random() * canvas.width;
        const y2 = acme.random() * canvas.height;
        ctx.moveTo(x1, y1);
        ctx.bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x2, y2);
      } else {
        const x2 = acme.random() * canvas.width;
        const y2 = acme.random() * canvas.height;
        ctx.moveTo(x1, y1);
        ctx.lineTo(x2, y2);
      }
      ctx.stroke();
    }
  });

```

---

## SVG Mode

SVG art doesn't need any JavaScript library. The ACME runtime is available if needed for seed-based randomness.

### Static SVG (No Runtime Needed)

```
<svg viewBox="0 0 400 400" xmlns="http://www.w3.org/2000/svg">
  <rect width="400" height="400" fill="#111"/>
  <g stroke="#FF4D94" stroke-width="1" fill="none">
    <circle cx="200" cy="200" r="50"/>
    <circle cx="200" cy="200" r="100"/>
    <circle cx="200" cy="200" r="150"/>
  </g>
</svg>
```

### Dynamic SVG with ACME Runtime

```
<!DOCTYPE html>
<html>
<head>
<style>
  body { margin: 0; background: #111; display: flex;
        justify-content: center; align-items: center; min-height: 100vh; }
  svg { max-width: 90vmin; max-height: 90vmin; }
</style>
</head>
<body>
<svg id="art" viewBox="0 0 400 400" xmlns="http://www.w3.org/2000/svg">
  <rect width="400" height="400" fill="#0a0a0a"/>
</svg>
<script>
acme.ready.then(() => {
  const svg = document.getElementById('art');
  const count = Math.floor(acme.random() * 20) + 10;

  for (let i = 0; i < count; i++) {
    const circle = document.createElementNS("http://www.w3.org/2000/svg", "circle");
    circle.setAttribute("cx", acme.random() * 400);
    circle.setAttribute("cy", acme.random() * 400);
    circle.setAttribute("r", acme.random() * 30 + 5);
    const hue = Math.floor(acme.random() * 60) + 300; // magenta range
    circle.setAttribute("fill", `hsla(${hue}, 70%, 60%, 0.5)`);
    circle.setAttribute("stroke", `hsl(${hue}, 70%, 80%)`);
```



```

        circle.setAttribute("stroke-width", "0.5");
        svg.appendChild(circle);
    }

    acme.trait("Circle Count", count);
});
</script>
</body>
</html>

```

---

## Traits and Attributes

Traits describe characteristics of the artwork for indexing, filtering, and display.

### Runtime Traits (Defined by Artist Code)

```

// String traits
acme.trait("Palette", "Cyber Neon")
acme.trait("Style", "Geometric")

// Numeric traits
acme.trait("Complexity", 7)
acme.trait("Element Count", shapes.length)

// Boolean traits (as strings for display)
acme.trait("Animated", "Yes")
acme.trait("Interactive", "No")

```

### Cortex Attributes (Defined at Mint Time)

These are set in the Cortex metadata when the asset is minted and cannot be changed by the artwork code:

```

{
  "attributes": [
    { "trait_type": "Artist", "value": "satoshi" },
    { "trait_type": "Collection", "value": "Genesis" },
    { "trait_type": "Edition", "value": 1 }
  ]
}

```

### Accessing Both

```

// Get everything merged
const traits = acme.getTraits()
// { "Artist": "satoshi", "Collection": "Genesis", "Palette": "Cyber Neon", ... }

```

```
// Access Cortex attributes directly
acme.attributes.forEach(attr => {
  console.log(`${attr.trait_type}: ${attr.value}`)
})
```

---

## Dynamic Component Import

Advanced technique for composable art — one piece can import code from another on-chain asset.

```
// Artist mints a utility library as "MYPALLETES"
// The library code (CommonJS):
//   module.exports = {
//     neonSunset: ['#FF4D94', '#FF6B35', '#FFD700'],
//     deepOcean:  ['#001F3F', '#0074D9', '#7FDBFF'],
//     pick: function(rng) { ... }
//   }

// Another artwork imports it:
const palettes = await acme.import("MYPALLETES");
const colors = palettes.pick(acme.random);
acme.trait("Palette", colors.name);
```

This enables a composable ecosystem where artists can share utility libraries, color palettes, easing functions, noise algorithms, etc. — all stored on-chain and loaded dynamically.

---

## Bootstrap Loader

The bootstrap loader is the ~500 byte script embedded in every minted HTML. It fetches and initializes the runtime.

### What Gets Minted (P5.js Art Example)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>ARTCORE Generative</title>
<script>(async function(){
var R="ACMERUNTIME010";
try{
```

```

var r=await fetch("/api/assets/"+encodeURIComponent(R)+"/content");
if(!r.ok)throw new Error("Runtime fetch: "+r.status);
var b=await r.arrayBuffer();
var v=new DataView(b);
var t;
if(v.byteLength>=2&&v.getUint8(0)===0x1f&&v.getUint8(1)===0x8b){
t=await new Response(new Blob([b]).stream().pipeThrough(new DecompressionStream("gzip"))).text();
}else{t=new TextDecoder().decode(b)}
var s=document.createElement("script");
s.textContent=t;
document.head.appendChild(s);
acme.ready.then(function(){return acme.loadLibrary("P5JSMIN190",{globalVar:"p5"})).catch(function(e){console.error("ACME bootstrap failed:",e)}})();
</script>
<style>
html, body { margin: 0; padding: 0; overflow: hidden; background: #000; }
canvas { display: block; }
</style>
</head>
<body>
<script>
// Artist code here - P5.js setup(), draw(), etc.
function setup() {
  createCanvas(windowWidth, windowHeight);
}
function draw() {
  background(0);
  fill(acme.random() * 255, 100, 200);
  circle(width/2, height/2, 100);
}
</script>
</body>
</html>

```

## Bootstrap Execution Sequence

1. Page loads, bootstrap script executes immediately
2. `fetch()` gets ACMERUNTIME010 content
3. Checks for gzip magic bytes (0x1f 0x8b) → decompresses if needed
4. Injects runtime as `<script>` → `window.acme` is defined
5. Runtime self-initializes: detects asset ID, resolves seed, sets up PRNG
6. `acme.ready` resolves
7. `acme.loadLibrary("P5JSMIN190")` fetches P5.js from on-chain, decompresses, injects
8. `window.p5` is available
9. P5.js finds `setup()` and `draw()` functions, begins execution

---

## Bridge Script (Preview Mode)

When previewing in the ARTCORE Studio, a different script is used instead of the bootstrap loader. The bridge script provides:

- **Console capture:** `console.log/warn/error/info` → `postMessage` to Studio
- **Error capture:** `window.onerror, unhandledrejection` → Studio console panel
- **FPS tracking:** `requestAnimationFrame` counter → Studio performance panel
- **Seeded PRNG:** Same `acme.hash/acme.random()` API as the runtime
- **Parameter API:** `acme.param()` for interactive controls
- **Seed updates:** Studio can push new seeds via `postMessage`
- **Screenshot capture:** `Canvas` → `toDataURL()` → Studio

The bridge script provides the same `window.acme` interface, so artist code works identically in preview and on-chain.

---

## Minting Flow

### From ARTCORE Studio to On-Chain

1. **Write code** in the Studio editor (P5.js, THREE.js, Canvas, SVG, or raw HTML)
2. **Preview** renders in the iframe using bridge script + on-chain library
3. **Final Code** panel shows the exact minted HTML (bootstrap loader + your code)
4. **Size budget** tracks the HTML size — must fit within Bitcoin witness limits
5. **Mint Art** navigates to the mint page with the generated HTML
6. **Asset creation** stores the HTML on-chain as an ACME asset
7. **Cortex metadata** defines the seed, traits, and generative-library tags

### Size Budget

| Component            | Typical Size       |
|----------------------|--------------------|
| Bootstrap loader     | ~500 bytes         |
| HTML wrapper         | ~300 bytes         |
| <b>Your code</b>     | Variable           |
| <b>Total must be</b> | <b>&lt; 399 KB</b> |

The runtime and libraries don't count — they're loaded from their own on-chain assets.

---

## Common Patterns

### Random Range Helper

```
function randomRange(min, max) {
  return min + acme.random() * (max - min);
}

function randomInt(min, max) {
  return Math.floor(randomRange(min, max));
}

function randomChoice(arr) {
  return arr[Math.floor(acme.random() * arr.length)];
}
```

### Weighted Random Selection

```
function weightedChoice(options) {
  // options: [{ value: "A", weight: 3 }, { value: "B", weight: 1 }]
  const total = options.reduce((sum, o) => sum + o.weight, 0);
  let r = acme.random() * total;
  for (const opt of options) {
    r -= opt.weight;
    if (r <= 0) return opt.value;
  }
  return options[options.length - 1].value;
}

const rarity = weightedChoice([
  { value: "Common",    weight: 60 },
  { value: "Uncommon",  weight: 25 },
  { value: "Rare",      weight: 10 },
  { value: "Legendary", weight: 5 },
]);
acme.trait("Rarity", rarity);
```

### Color Palette from Seed

```
function generatePalette(count) {
  const baseHue = acme.random() * 360;
  const spread = acme.random() * 60 + 20;
```

```

    return Array.from({ length: count }, (_, i) => {
      const hue = (baseHue + i * spread / count + acme.random() * 10) % 360;
      const sat = 60 + acme.random() * 30;
      const lit = 40 + acme.random() * 30;
      return `hsl(${hue}, ${sat}%, ${lit}%)`;
    });
  }

  const palette = generatePalette(5);

```

### Gaussian Random (Box-Muller)

```

function randomGaussian(mean, stddev) {
  const u1 = acme.random();
  const u2 = acme.random();
  const z = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
  return mean + z * stddev;
}

```

---

## Troubleshooting

### “acme is not defined”

The runtime hasn’t loaded yet. If using the bootstrap loader, make sure your code runs after the runtime is injected. Use `acme.ready`:

```

// Wrong - acme might not exist yet
console.log(acme.hash);

// Right - wait for ready
acme.ready.then(() => {
  console.log(acme.hash);
});

```

### “acme.random() called before ready”

You’re calling `acme.random()` before the seed has been resolved. Wrap your initialization in `acme.ready.then(...)`.

### P5.js/THREE.js not loading

The library assets (P5JSMIN190, THREEJSMIN01580) must be minted on-chain. Check that: 1. The asset exists: `curl /api/assets/P5JSMIN190/content` should return JavaScript 2. The content is valid (may be gzipped — the runtime handles decompression automatically) 3. The global variable name matches (p5 for P5.js, THREE for THREE.js)

## Different output each load

You're using `Math.random()` or `random()` (P5's) instead of `acme.random()`. Also check for `Date.now()`, `new Date()`, or `performance.now()` in your initialization code.

## Art doesn't resize

Add a resize handler: - **P5.js**: `function windowResized() { resizeCanvas(windowWidth, windowHeight); }` - **THREE.js**: `window.addEventListener('resize', () => { camera.aspect = ...; renderer.setSize(...); })` - **Canvas**: `window.addEventListener('resize', () => { canvas.width = ...; })`

---

## Full Runtime Source Code

This is the complete, unminified ACME Runtime source — exactly what gets minted as `ACMERUNTIME010` (after minification).

```
(function(){  
  "use strict";  
  
  var ACME_RUNTIME_VERSION="1.0.0";  
  
  //   SFC32 PRNG  
  // Small Fast Counting (SFC32) pseudo-random number generator.  
  // Same algorithm used by f3hash and Art Blocks.  
  // Takes a 64-char hex seed, returns a function that produces  
  // deterministic floats in [0, 1).  
  function sfc32(h){  
    var a=parseInt(h.slice(2,10),16)|0,  
        b=parseInt(h.slice(10,18),16)|0,  
        c=parseInt(h.slice(18,26),16)|0,  
        d=parseInt(h.slice(26,34),16)|0;  
    return function(){  
      a|=0;b|=0;c|=0;d|=0;  
      var t=(a+b|0)+d|0;  
      d=d+1|0;  
      a=b^(b>>>9);  
      b=c+(c<<3)|0;  
      c=(c<<21)|(c>>>11);  
      c=c+t|0;  
      return(t>>>0)/4294967296;  
    };  
  }  
}
```

```

// SHA-256 for seed derivation
// Used to derive deterministic seeds for subassets (e.g., COLLECTION.42).
// Returns a hex string like "0xabcd...".
async function sha256hex(str){
  var buf=new TextEncoder().encode(str);
  var hash=await crypto.subtle.digest("SHA-256",buf);
  return"0x"+Array.from(new Uint8Array(hash),function(b){
    return b.toString(16).padStart(2,"0")
  }).join("");
}

// Random seed fallback
// Generates a cryptographically random 32-byte hex seed.
// Only used when no deterministic seed source is available.
function randomSeed(){
  return"0x"+Array.from(crypto.getRandomValues(new Uint8Array(32)),function(b){
    return b.toString(16).padStart(2,"0")
  }).join("");
}

// Asset ID Detection
// Detects the current asset ID from the serving URL.
// Looks for the pattern /api/assets/{ASSET_ID}/content in the path,
// then falls back to ?asset= query parameter.
function detectAssetId(){
  var m=location.pathname.match(/\/api\/assets\/([~\/]+)\/content/);
  if(m)return decodeURIComponent(m[1]);
  var u=new URLSearchParams(location.search);
  return u.get("asset")||null;
}

// Parse asset name into parent/child/tokenNumber
// ACME assets can be hierarchical: "PARENT.CHILD"
// For example, "MYART.42" → parent="MYART", child="42", tokenNumber=42
function parseAssetName(assetId){
  if(!assetId)return{parent:null,child:null,tokenNumber:null};
  var dot=assetId.indexOf(".");
  if(dot===-1)return{parent:assetId,child:null,tokenNumber:null};
  var parent=assetId.slice(0,dot);
  var child=assetId.slice(dot+1);
  var num=parseInt(child,10);
  return{parent:parent,child:child,tokenNumber:isNaN(num)?null:num};
}

// ACME Object
// The main runtime object exposed as window.acme

```



```

var ACME={
  // Core
  hash:null,           // Deterministic seed (0x...)
  _prng:null,          // SFC32 instance (lazy-initialized)
  attributes:[],       // NFT attributes from Cortex metadata
  _traits:{},          // Runtime-defined traits

  // Asset Identity
  assetId:detectAssetId(),
  tokenNumber:null,
  childName:null,
  parentAsset:null,

  // Ready Promise
  ready:null,
  _readyResolve:null,

  // Seeded PRNG
  // Returns deterministic float in [0, 1) based on acme.hash
  random:function(){
    if(!ACME._prng){
      if(!ACME.hash)throw new Error("acme.random() called before ready");
      ACME._prng=sfc32(ACME.hash);
    }
    return ACME._prng();
  },

  // Trait System
  trait:function(name,value){
    ACME._traits[name]=value;
  },

  getTraits:function(){
    var all={};
    // Start with Cortex attributes
    ACME.attributes.forEach(function(a){
      all[a.trait_type]=a.value;
    });
    // Override/extend with runtime traits
    for(var k in ACME._traits){
      if(ACME._traits.hasOwnProperty(k))all[k]=ACME._traits[k];
    }
    return all;
  },

  // Dynamic import (CommonJS → ES module)

```

```

// Imports another on-chain asset as a module.
// The asset's code is wrapped in a CommonJS → ES module adapter.
_importCache:{},
_importing:{},
import:function(assetId){
  if(ACME._importCache[assetId])return Promise.resolve(ACME._importCache[assetId]);
  if(ACME._importing[assetId])return ACME._importing[assetId];
  var p=fetch("/api/assets/"+encodeURIComponent(assetId)+"/content")
  .then(function(r){
    if(!r.ok)throw new Error("Component not found: "+assetId);
    return r.text()
  })
  .then(function(src){
    var wrapped="var module={exports:{};var exports=module.exports;"
      +src+";export default module.exports;";
    var b=new Blob([wrapped],{type:"application/javascript"});
    var u=URL.createObjectURL(b);
    return import(u).then(function(mod){
      URL.revokeObjectURL(u);
      var resolved=mod.default||mod;
      ACME._importCache[assetId]=resolved;
      delete ACME._importing[assetId];
      return resolved
    });
  })
  .catch(function(e){
    delete ACME._importing[assetId];
    console.error("acme.import failed for "+assetId+":",e);
    throw e
  });
  ACME._importing[assetId]=p;
  return p;
},

// Library loading (gzip decompress + script inject)
// Loads a library from on-chain, decompresses if gzipped,
// and injects as a <script> tag.
_libCache:{},
_libLoading:{},
loadLibrary:function(assetOrUrl,opts){
  opts=opts||{};
  var key=assetOrUrl;
  if(ACME._libCache[key])return Promise.resolve(ACME._libCache[key]);
  if(ACME._libLoading[key])return ACME._libLoading[key];

  // If it looks like a URL (contains /), use directly; otherwise build asset URL

```

```

var url=assetOrUrl.indexOf("/")>=0
? assetOrUrl
: "/api/assets/"+encodeURIComponent(assetOrUrl)+"/content";

var p=fetch(url)
.then(function(r){
  if(!r.ok)throw new Error("Library fetch failed: "+r.status);
  return r.arrayBuffer()
})
.then(function(buf){
  // Check for gzip magic bytes (0x1f 0x8b)
  var v=new DataView(buf);
  if(v.byteLength>=2&&v.getUint8(0)===0x1f&&v.getUint8(1)===0x8b){
    // Decompress gzip using browser's built-in DecompressionStream
    var ds=new DecompressionStream("gzip");
    return new Response(new Blob([buf]).stream().pipeThrough(ds)).text();
  }
  // Not gzipped - decode as UTF-8
  return new TextDecoder().decode(buf);
})
.then(function(src){
  // Inject as <script> tag
  var s=document.createElement("script");
  s.textContent=src;
  document.head.appendChild(s);

  // Return the global variable if specified
  var globalVar=opts.globalVar;
  var result=globalVar?window[globalVar]:true;
  ACME._libCache[key]=result;
  delete ACME._libLoading[key];
  return result;
})
.catch(function(e){
  delete ACME._libLoading[key];
  console.error("acme.loadLibrary failed for "+key+":",e);
  throw e;
});

ACME._libLoading[key]=p;
return p;
}
};

// Parse asset name structure (parent/child/tokenNumber)
var parsed=parseAssetName(ACME.assetId);

```

```

ACME.parentAsset=parsed.parent;
ACME.childName=parsed.child;
ACME.tokenNumber=parsed.tokenNumber;

// Create ready promise
ACME.ready=new Promise(function(resolve){
    ACME._readyResolve=resolve;
});

// Initialization
// Resolves the seed through the priority chain:
// 1. URL params (?hash=, ?seed=, ?tokenHash=)
// 2. Cortex metadata tag (x:generative-seed)
// 3. SHA-256 derivation from asset name (for subassets)
// 4. Random fallback
async function init(){
    // 1. Check URL params first (highest priority)
    var params=new URLSearchParams(location.search);
    var urlSeed=params.get("hash")||params.get("seed")||params.get("tokenHash");

    if(urlSeed){
        ACME.hash=urlSeed;
        ACME._readyResolve(ACME);
        return;
    }

    // 2. Try to fetch Cortex metadata for seed + attributes
    if(ACME.assetId){
        try{
            var r=await fetch("/api/assets/"+encodeURIComponent(ACME.assetId)+"/cortex");
            if(r.ok){
                var data=await r.json();
                if(data.result){
                    var tags=data.result.tags||[];
                    var seedTag=tags.find(function(t){return t.tag=="x:generative-seed"});
                    if(seedTag&&seedTag.value){
                        ACME.hash=seedTag.value;
                    }
                    ACME.attributes=data.result.attributes||[];
                }
            }
        }catch(e){
            console.warn("Cortex metadata fetch failed:",e.message);
        }
    }
}

```

```

// 3. Derive seed from child number (for subassets like ART.42)
if(!ACME.hash&&ACME.childName){
  try{
    ACME.hash=await sha256hex(ACME.assetId);
  }catch(e){
    console.warn("SHA-256 derivation failed:",e.message);
  }
}

// 4. Random fallback
if(!ACME.hash){
  ACME.hash=randomSeed();
}

ACME._readyResolve(ACME);
}

// Expose globally
window.acme=ACME;
window.ACME_RUNTIME_VERSION=ACME_RUNTIME_VERSION;

// Initialize - with error recovery
init().catch(function(e){
  console.error("ACME runtime init failed:",e);
  if(!ACME.hash)ACME.hash=randomSeed();
  ACME._readyResolve(ACME);
});
})();

```

---

## Version History

| Version | Asset          | Changes                                                                                                               |
|---------|----------------|-----------------------------------------------------------------------------------------------------------------------|
| 1.0.0   | ACMERUNTIME010 | Initial release: SFC32 PRNG, seed resolution chain, trait system, library loading, component import, subasset support |